RSA 타이밍 부채널 실험 요약

2차 실험 결과 · 업로드된 timing_results_1.csv 분석

1. 실험 목적

비밀 지수의 비트 패턴에 따라 RSA 모듈러 거듭제곱 연산의 실행 시간이 달라질 수 있는지 관찰하는 실험이다. 특히 지수의 1 비트 개수인 Hamming Weight(HW)와 평균 실행 시간 사이의 관계를 분석했다. 이번 실험은 교육용 모듈러 거듭제곱 코드의 시간 누출 관찰이며, 실제 RSA 개인키 복구를 수행한 실험은 아니다.

2. 실험 설정

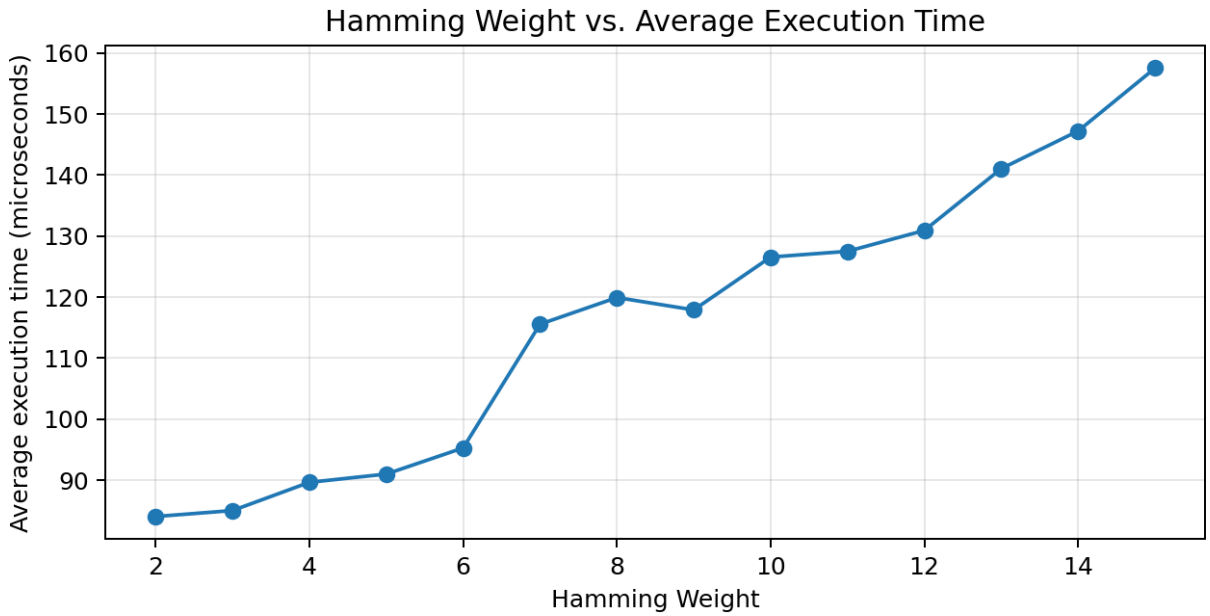
항목	설정
모듈러스	modulus = (1 << 2048) - 159
기준값	base = 123456789
측정 함수	time.perf_counter_ns()
지수별 반복 횟수	1,000회
분석 대상	서로 다른 지수 14개 (HW 2~15)

3. 주요 측정 결과

지표	결과
측정 지수 수	14개
평균 실행 시간(지수별 평균의 평균)	116.37 µs
최소 지수별 평균	84.05 µs (HW=2)
최대 지수별 평균	157.51 µs (HW=15)
Pearson 상관계수	0.9825
Pearson p-value	3.963e-10
Spearman 순위상관계수	0.9956
Spearman p-value	1.032e-13

Pearson $r = 0.9825$, Spearman $\rho = 0.9956$로, 이번 데이터에서는 Hamming Weight가 커질수록 평균 실행 시간이 증가하는 강한 양의 관계가 관찰됐다. 두 검정의 p-value도 작게 나타나 관찰된 연관성이 통계적으로 뚜렷하다. 다만 분석 단위는 14개 지수별 평균값이므로, 이를 곧바로 실제 키 복구 성공이나 일반적인 RSA 구현의 취약성 증명으로 해석해서는 안 된다.

4. Hamming Weight와 실행 시간



그래프는 CSV의 지수별 평균 실행 시간을 마이크로초 단위로 나타낸다. 전체적인 증가 추세가 보이지만 모든 인접 지점이 단조롭게 증가하는 것은 아니다. 개별 측정에는 시스템 스케줄링, 인터프리터 및 실행 환경의 잡음이 포함될 수 있다.

5. 결과 해석과 한계

해석: 실험 코드의 Square - and - Multiply 구조에서는 지수 비트가 1일 때 추가 곱셈이 수행된다. 따라서 Hamming Weight가 큰 지수일수록 추가 연산이 많아지고, 이번 큰 정수 연산 환경에서는 그 차이가 평균 실행 시간에 반영된 것으로 해석할 수 있다.

한계: 사용한 modulus 값은 두 소수의 곱으로 생성한 실제 RSA 모듈러스가 아니다. 또한 Python 타이밍 측정은 운영체제 스케줄링, CPU 상태, 인터프리터 오버헤드 등의 영향을 받는다. 이번 분석은 지수 전체 복구를 수행하지 않았으며, 특정 실험 코드에서의 시간 의존성을 관찰한 것이다.

6. 다음 단계

1) 현재 코드를 취약 구현의 기준선으로 보존한다.

2) Square - and - Multiply - Always 형태의 대응 실험 코드를 구현한다.

3) 동일한 지수, 입력값, 모듈러스 및 측정 절차로 두 구현을 비교한다.

4) 실행 순서를 무작위화하고 여러 독립 배치로 반복 측정한다.

5) Hamming Weight와 실행 시간의 상관관계가 대응 구현에서 줄어드는지 분석한다.

6) 대응 구현도 Python 자체가 완전한 constant - time을 보장하지 않는다는 점을 README에 명시한다.

7. 포트폴리오용 요약 문장

Python으로 구현한 모듈러 거듭제곱 연산을 대상으로 실행 시간을 측정하고, 비밀 지수의 Hamming Weight와 실행 시간 사이의 관계를 통계적으로 분석했다. 2차 실험에서 강한 양의 상관관계를 관찰했으며, 향후 고정 연산 구조의 대응 알고리즘과 비교해 시간 누출 완화 효과를 검증할 예정이다.

분석 출처: 사용자가 제공한 timing_results_1.csv. 본 문서는 해당 CSV의 요약이며, 실제 측정 결과와 해석의 범위를 구분해 작성했다.